

Run it in a browser (GitHub Codespaces or Gitpod)

Milestone 1.1 is the data + logic foundation (no screen of its own). **Milestone 1.2 adds the booking screen** — a reference widget that walks a patient through doctor → date/time → details → a live held slot with a countdown. That widget is the thing to show the client.

Replace `<OWNER>/<REPO>` below with your repository once it's pushed.

Option A — GitHub Codespaces (recommended)

1. Push this repo to GitHub (see the repo's push instructions).
2. Open `https://codespaces.new/<OWNER>/<REPO>` — or on the repo page: **Code** ▶ **Codespaces** ▶ **Create codespace on main**.
3. Wait for the container to build. The `.devcontainer` automatically: installs deps, generates the Prisma client, **applies the migrations**, and **seeds** a demo clinic. (First build takes a couple of minutes.)

Option B — Gitpod

Open `https://gitpod.io/#https://github.com/<OWNER>/<REPO>`. The `.gitpod.yml` runs `install` ▶ `generate` ▶ `migrate` ▶ `seed`, then `npm test`.

See the booking screen (the client-facing visual)

In a Codespace / Gitpod (uses the real Express + Prisma app):

```
npm run dev
```

Then open the forwarded **port 3000** (Codespaces pops a "Open in Browser" prompt). You'll see the booking widget backed by the real API.

On any machine with Postgres but no Prisma (e.g. the original Windows box):

```
npm run preview      # pg-backed demo server, seeds a doctor + 14 days of slots
# open http://localhost:3000
```

Either way you get: pick a doctor → pick a date & time → enter details → the slot is **held with a live countdown** (the M1.2 reservation TTL), and the payment button shows that Pix + card land in Milestone 1.3.

Prove the guarantees in the terminal (Milestone 1.1)

Once the environment is ready, run these in the built-in terminal:

```
# 1. Unit tests – state machine + slot materialization math (12 tests)
npm test

# 2. Full typecheck
npm run typecheck

# 3. DB-level constraint proof (overlap + duplicate rejected, valid accepted)
psql "$DATABASE_URL" -f scripts/verify-m11-constraints.sql

# 4. The concurrency + expiry guarantees, live against Postgres
npm run verify:locking      # 50 concurrent bookings on one slot -> exactly 1 wins
npm run verify:expiry       # expired -> freed, not-due -> held, paid -> never freed
```

Everything above is Milestone 1.1 (plus the M1.2 concurrency/expiry guarantees it enables). To browse the data visually:

```
npx prisma studio          # opens a DB GUI (forwarded to your browser)
```

What you should see

- `npm test` → **12 passed**.
- The constraint proof → four `PASS` lines (overlap rejected, duplicate rejected, adjacent slot accepted, final count = 2).
- `verify:locking` → 20 rounds, each **exactly 1 success / 49 clean losses**.

Note: unlike this project's original Windows dev box, the cloud Linux runtime runs Prisma's engine, so the Express API (`npm run dev`) and the Prisma integration tests (`npm run test:integration`) also work here.